

运动控制（ROS2）

本文档描述通过 ROS2 话题控制山猫 M20 运动状态、步态和速度的方法。与 [basic server 运动控制](#) 相比，ROS2 方式需要开发者手动管理运动状态机，适合机载算法开发与自定义导航场景。

前置阅读：[对外通信方式](#)。

目录

- 1. 运动状态定义
- 2. 步态定义
- 3. 状态转换逻辑
- 4. ROS2 话题
 - 4.1 运动状态转换: /MOTION_STATE
 - 4.2 步态切换: /GAIT
 - 4.3 速度指令: /NAV_CMD
 - 4.4 运动基本状态: /MOTION_INFO
- 5. 开发建议

1. 运动状态定义

运动控制使用以下统一的状态值：

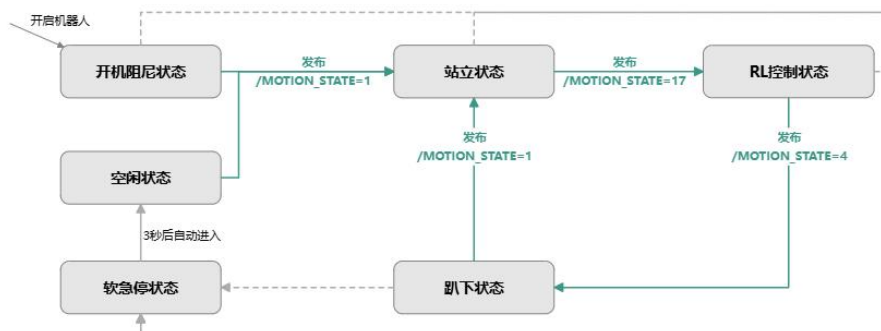
十六进制值	十进制值	名称	说明
0x0	0	空闲	空闲状态，无动作，等待指令
0x1	1	站立	临时过渡状态，会自动跳转到RL控制状态
0x2	2	软急停	关节断电，最高优先级，任意状态下均可触发
0x3	3	开机阻尼	开机后自动进入的阻尼状态
0x4	4	趴下	安全趴下状态
0x11	17	RL 控制	强化学习控制模式， 唯一可执行移动和步态切换的状态

2. 步态定义

十六进制值	十进制值	名称	运动模式	适用场景
0x1001	4097	基础	标准	室内平坦地面，手动遥控
0x1003	4099	楼梯	标准	台阶/路沿，手动遥控
0x3002	12290	平地	敏捷	自主导航 + 平地行走
0x3003	12291	楼梯	敏捷	自主导航 + 台阶跨越

… 敏捷运动模式下的步态速度响应性能较好，适用于导航等自主算法开发。

3. 状态转换逻辑



调用逻辑建议：

- **站立状态**为一个临时状态，不能进行移动控制操作；仅在**RL控制状态**下可切换步态或执行运动控制轴指令和速度指令。
- 因此使用ROS话题让机器人起立时，需要遵循以下步骤：
 - 检查机器人是否处于开机阻尼状态/空闲状态/趴下状态：若处于这些状态下，可向话题/MOTION_STATE发布消息字段state=1；若机器人姿态正常，机器人将切换到站立状态；若姿态异常，机器人将拒绝起立并上报错误。
 - 机器人进入站立状态后，需要开发者向话题/MOTION_STATE发布消息字段state=17，使机器人进入RL控制状态，此时才算真正完成起立，使机器人处于可接收速度控制指令下发的状态下。
- 使用ROS话题让机器人趴下时，需注意，机器人仅在处于RL控制状态且静止时执行趴下指令，若机器人正在运动中，不应下发趴下指令。
- 软急停指令为最高优先级，任意状态下均可下发生效。
- 起立/趴下过程中，除软急停外，其余运动控制操作均被禁用。

与 basic_server 协议的差异

维度	basic_server 协议	ROS2
运动状态机	自动状态推进（站立后自动进入空闲）	手动管理：需逐步发布状态值

速度指令	Type=2 Cmd=21 (常规/辅助) 或 Cmd=25 (导航)	<code>/NAV_CMD</code> 话题
步态切换	Type=2 Cmd=23	<code>/GAIT</code> 话题
状态反馈	Type=1002 Cmd=4/6 (主动推送)	<code>/MOTION_INFO</code>
急停	Type=2 Cmd=22 MotionParam=2	发布 state=2 到 <code>/MOTION_STATE</code>
适用场景	远程集成、APP 开发	机载算法、自定义导航

💡 `basic_server` 协议的状态机有自动推进机制：**站立 (1)** 后自动进入**RL控制 (17)**，随后可发送速度指令运动。

ROS2 方式下，状态转换需要开发者**逐步显式控制**，以执行起立为例，需要先发布 `state=1` 进入站立，等待机器人完成站立动作后，再发布 `state=17` 进入 RL 控制。只有进入 RL 控制后，机器人才能接收速度指令。

4. ROS2 话题

4.1 运动状态转换: `/MOTION_STATE`

该话题在运动程序启动后激活，开发者可在该话题中发布消息，以转换机器人运动状态。

属性	值
消息类型	drdds/msg/MotionState
方向	发布（客户端 → 机器人）

消息结构

▼

复制代码

1

MetaType header

2

uint64 frame_id

3

Timestamp timestamp

4

int32 sec

5

uint32 nsec

6

MotionStateValue data

7

int32 state

state发布值（十进制）	state发布值（十六进制）	目标状态
1	0x1	站立
2	0x2	软急停
4	0x4	趴下
17	0x11	RL 控制

...

下发控制指令后，请通过 `/MOTION_INFO` 的订阅反馈确认状态是否已切换到目标值。
开发者如需在自己的程序中订阅该话题，需要在程序中定义该自定义消息的结构。

4.2 步态切换: /GAIT

该话题在运动程序启动后激活，开发者可在该话题中发布消息，以切换运控步态。

属性	值
消息类型	drdds/msg/Gait
方向	发布（客户端 → 机器人）

消息结构

▼

复制代码

```
1  MetaType header
2      uint64 frame_id
3      Timestamp timestamp
4          int32 sec
5          uint32 nsec
6  GaitValue data
7      uint32 gait
```

gait发布值（十进制）	gait发布值（十六进制）	目标步态
4097	0x1001	基础（标准运动模式）
4099	0x1003	楼梯（标准运动模式）
12290	0x3002	平地（敏捷运动模式）
12291	0x3003	楼梯（敏捷运动模式）

- 步态切换**仅在 RL 控制状态下**执行，且必须等前一步态切换完成后才能下发下一个。
- 每次重新进入 RL 控制状态时，建议先手动下发重置步态为基础步态（0x1001），再切换到目标步态。
- 步态切换仅在**静止状态**下允许。

...

下发控制指令后，请通过 `/MOTION_INFO` 的订阅反馈确认步态是否已切换为目标值。
开发者如需在自己的程序中订阅该话题，需要在程序中定义该自定义消息的结构。

4.3 速度指令：/NAV_CMD

开发者可通过发布该话题来下发导航规划的速度指令，以控制机器人移动；也可订阅话题消息，以获取机器人本身导航功能下发的速度指令。

属性	值
消息类型	drdds/msg/NavCmd
方向	发布 / 订阅
频率建议	10Hz
使用模式	仅导航模式

消息结构

▼

复制代码

1

MetaType header

2

uint64 frame_id

3

Timestamp timestamp

4

int32 sec

5

uint32 nsec

6

NavCmdValue data

7

float32 x_vel

8

float32 y_vel

9

float32 yaw_vel

字段	类型	单位	范围	说明
x_vel	float32	m/s	视步态而定	正=前进，负=后退
y_vel	float32	m/s	视步态而定	正=左移，负=右移
yaw_vel	float32	rad/s	视步态而定	正=逆时针，负=顺时针

.....

- 该速度话题**仅在导航模式下生效**，建议以 10Hz 的固定频率下发。
- 发布该话题时，可能会与机器人自带的 `planner` 服务产生冲突，需关闭 `planner` 服务。同时该话题依赖 `basic_server` 服务，若速度指令无响应，请检查服务存活状态。
- 发布该话题时，可能会与自主充电程序所使用的 `charge_manager` 服务产生冲突，请避免在机器人执行自主充电任务（自主充电未处于空闲状态）时发布该话题。

💡 开发者如需在自己的程序中订阅该话题，需要在程序中定义该自定义消息的结构。

各个步态的有效速度范围

- 适用范围：软件包版本 $\geq$ V1.1.7

步态名称	十六进制码	X轴线速度(m/s)	Y轴线速度(m/s)	Yaw角速度(rad/s)
标准-基础	0x1001	$[-2.0, -0.2] \cup [0.2, 2.0]$	$[-1.0, -0.35] \cup [0.35, 1.0]$	$[-2.0, -0.5] \cup [0.5, 2.0]$
标准-楼梯	0x1003	$[-2.0, -0.15] \cup [0.15, 2.0]$	$[-1.0, -0.3] \cup [0.3, 1.0]$	$[-2.0, -0.4] \cup [0.4, 2.0]$
敏捷-平地	0x3002	$[-2.0, -0.15] \cup [0.15, 2.0]$	$[-1.0, -0.25] \cup [0.25, 1.0]$	$[-1.5, -0.35] \cup [0.35, 1.5]$
敏捷-楼梯	0x3003	$[-2.0, -0.15] \cup [0.15, 2.0]$	$[-1.0, -0.3] \cup [0.3, 1.0]$	$[-2.0, -0.4] \cup [0.4, 2.0]$

4.4 运动基本状态: /MOTION_INFO

属性	值
消息类型	drdds/msg/MotionInfo
方向	订阅 (机器人 → 客户端)
频率	20Hz

`/MOTION_INFO` 话题在开机后即有消息发布，包含运动状态、步态、速度和续航等综合信息，是 ROS2 方式下获取运动状态的主要话题。

消息结构

▼复制代码

```
1  MetaType header
2      uint64 frame_id
3      Timestamp timestamp
4      int32 sec
5      uint32 nsec
6  MotionInfoValue data
7      float32 vel_x      # 前后方向运动速度 (m/s)
8      float32 vel_y      # 左右方向运动速度 (m/s)
9      float32 vel_yaw    # 偏航角速度 (rad/s)
10     float32 height      # 身体高度 (m)
11     MotionStateValue data # 运动状态
12         int32 state
13     GaitValue data      # 步态
14         uint32 gait
15     float32 payload      # 负载 (无效参数)
16     float32 remain_mile  # 剩余续航里程 (km)
```

字段	类型	单位	说明
vel_x	float32	m/s	正=前进，负=后退
vel_y	float32	m/s	正=左移，负=右移
vel_yaw	float32	rad/s	正=逆时针，负=顺时针
state	int32	—	参数值含义请参考第1节
gait	uint32	—	参数值含义请参考第2节

🗨 开发者如需在自己的程序中订阅该话题，需要在程序中定义该自定义消息的结构。

5. 开发建议

1. **优先使用 basic_server 协议进行运动控制：**basic_server 协议内置状态机管理，调用更简单、更安全。除非需要在机载开发自定义运动算法，否则建议使用 [basic_server 运动控制](#)。
2. **ROS2 适合算法开发：**如果需要在机器人上运行自定义导航、SLAM 或运动规划算法，通过 ROS2 可以直接获取传感器数据。
3. **混合使用：**可以同时使用两种方式——用 basic_server 协议管理设备状态和任务，用 ROS2 获取传感器数据驱动算法。
4. **网段选择：**确保开发主机接入正确的网段以获取所需话题。在 31 和 33 网段上，运动控制相关话题均可订阅。
5. **超时保护：**如果超过 500ms 未收到速度指令，机器人会自动减速停止。保持 20Hz 的发布频率。

📄 相关文档：[ROS 2 / DDS 接口总览](#) | [ROS 2 / DDS 话题速查](#)

4.4 运动基本状态: /MOTION_INFO

属性	值
消息类型	drdds/msg/MotionInfo
方向	订阅 (机器人 → 客户端)
频率	20Hz

`/MOTION_INFO` 话题在开机后即有消息发布，包含运动状态、步态、速度和续航等综合信息，是 ROS2 方式下获取运动状态的主要话题。

消息结构

▼复制代码

```
1  MetaType header
2      uint64 frame_id
3      Timestamp timestamp
4      int32 sec
5      uint32 nsec
6  MotionInfoValue data
7      float32 vel_x      # 前后方向运动速度 (m/s)
8      float32 vel_y      # 左右方向运动速度 (m/s)
9      float32 vel_yaw    # 偏航角速度 (rad/s)
10     float32 height      # 身体高度 (m)
11     MotionStateValue data # 运动状态
12         int32 state
13     GaitValue data      # 步态
14         uint32 gait
15     float32 payload      # 负载 (无效参数)
16     float32 remain_mile  # 剩余续航里程 (km)
```